Smart Waste Management System - ESP32 Controller

Overview

This sketch connects to the Node.js backend via Socket.IO and controls the conveyor belt based on dashboard commands.

Required Libraries

Install via Arduino Library Manager:

1. **WebSockets** by Markus Sattler (arduinoWebSockets)
2. **ArduinoJson** by Benoit Blanchon

Wiring

Pin	Purpose
GPIO 25	Motor Control (PWM for speed)
GPIO 34	IR Sensor (detect items)
GPIO 2	LED Status (built-in)

Bin Mapping

Index	Category
0	Recyclable
1	Wet Waste
2	Hazardous
3	Dry Waste

Complete Code

```
#include <WiFi.h>
#include <WebSocketsClient.h>
#include <ArduinoJson.h>

// ============================================================
// CONFIGURATION - CHANGE THESE VALUES
// ============================================================

const char* WIFI_SSID = "YOUR_WIFI_SSID";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
const char* SERVER_IP = "192.168.1.100";
const int SERVER_PORT = 3001;
```

```
#define MOTOR_PIN 25
#define IR_SENSOR_PIN 34
#define LED_PIN 2

#define SPEED_SLOW 80
#define SPEED_MEDIUM 150
#define SPEED_FAST 255
#define ITEM_DEBOUNCE_MS 2000

// ============================================================
// GLOBAL VARIABLES
// ============================================================

WebSocketsClient webSocket;
bool systemPowerOn = false;
String currentSpeed = "medium";
int currentPWM = SPEED_MEDIUM;
unsigned long lastItemTime = 0;
int lastBinType = -1;
bool isConnected = false;

// ============================================================
// SETUP
// ============================================================

void setup() {
    Serial.begin(115200);
    Serial.println("\n=== Smart Waste Management ESP32 ===");

    pinMode(MOTOR_PIN, OUTPUT);
    pinMode(IR_SENSOR_PIN, INPUT);
    pinMode(LED_PIN, OUTPUT);

    analogWrite(MOTOR_PIN, 0);
    digitalWrite(LED_PIN, LOW);

    connectWiFi();
    setupSocketIO();
}

// ============================================================
// MAIN LOOP
// ============================================================

void loop() {
    webSocket.loop();
    checkItemSensor();

    if (isConnected) {
        static unsigned long lastBlink = 0;
        if (millis() - lastBlink > 1000) {
            digitalWrite(LED_PIN, !digitalRead(LED_PIN));
            lastBlink = millis();
```

```
        }
    }
    delay(10);
}

// ============================================================
// WIFI CONNECTION
// ============================================================

void connectWiFi() {
    Serial.printf("Connecting to WiFi: %s", WIFI_SSID);
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    int attempts = 0;
    while (WiFi.status() != WL_CONNECTED && attempts < 30) {
        delay(500);
        Serial.print(".");
        attempts++;
    }

    if (WiFi.status() == WL_CONNECTED) {
        Serial.println("\nWiFi Connected!");
        Serial.print("IP: ");
        Serial.println(WiFi.localIP());
        digitalWrite(LED_PIN, HIGH);
    } else {
        Serial.println("\nWiFi Failed! Restarting...");
        delay(5000);
        ESP.restart();
    }
}

// ============================================================
// SOCKET.IO SETUP
// ============================================================

void setupSocketIO() {
    String path = "/socket.io/?EIO=4&transport=websocket";
    webSocket.begin(SERVER_IP, SERVER_PORT, path.c_str());
    webSocket.onEvent(webSocketEvent);
    webSocket.setReconnectInterval(5000);
    Serial.printf("Connecting to: %s:%d\n", SERVER_IP, SERVER_PORT);
}

// ============================================================
// WEBSOCKET EVENT HANDLER
// ============================================================

void webSocketEvent(WStype_t type, uint8_t* payload, size_t length) {
    switch (type) {
        case WStype_DISCONNECTED:
            Serial.println("Disconnected");
            isConnected = false;
```

```
            digitalWrite(LED_PIN, LOW);
            setMotorSpeed(0);
            break;
        case WStype_CONNECTED:
            Serial.println("Connected!");
            isConnected = true;
            digitalWrite(LED_PIN, HIGH);
            break;
        case WStype_TEXT:
            handleSocketMessage((char*)payload);
            break;
        default:
            break;
    }
}

// ============================================================
// SOCKET MESSAGE HANDLER
// ============================================================

void handleSocketMessage(char* payload) {
    if (payload[0] != '4' || payload[1] != '2') {
        if (payload[0] == '2') webSocket.sendTXT("3");
        return;
    }

    char* jsonStart = payload + 2;
    StaticJsonDocument<512> doc;
    DeserializationError error = deserializeJson(doc, jsonStart);
    if (error) return;

    const char* eventName = doc[0];

    if (strcmp(eventName, "esp_control") == 0) {
        handleControlCommand(doc[1]);
    }
    else if (strcmp(eventName, "ai_inference") == 0) {
        handleAIInference(doc[1]);
    }
}

// ============================================================
// CONTROL COMMAND HANDLER
// ============================================================

void handleControlCommand(JsonObject data) {
    const char* command = data["command"];

    if (strcmp(command, "power") == 0) {
        systemPowerOn = data["value"].as<bool>();
        Serial.printf("Power: %s\n", systemPowerOn ? "ON" : "OFF");
        setMotorSpeed(systemPowerOn ? currentPWM : 0);
    }
    else if (strcmp(command, "speed") == 0) {
```

```
        currentSpeed = data["value"].as<String>();
        Serial.printf("Speed: %s\n", currentSpeed.c_str());

        if (currentSpeed == "slow") currentPWM = SPEED_SLOW;
        else if (currentSpeed == "medium") currentPWM = SPEED_MEDIUM;
        else if (currentSpeed == "fast") currentPWM = SPEED_FAST;

        if (systemPowerOn) setMotorSpeed(currentPWM);
    }
}

// ============================================================
// AI INFERENCE HANDLER
// ============================================================

void handleAIInference(JsonObject data) {
    int labelId = data["label_id"].as<int>();
    float confidence = data["confidence"].as<float>();

    if (confidence > 60.0 && labelId >= 0 && labelId <= 3) {
        lastBinType = labelId;
        Serial.printf("AI: Bin %d (%.1f%%)\n", labelId, confidence);
    }
}

// ============================================================
// ITEM SENSOR CHECK
// ============================================================

void checkItemSensor() {
    bool itemDetected = digitalRead(IR_SENSOR_PIN) == LOW;
    unsigned long now = millis();

    if (itemDetected &&
        (now - lastItemTime > ITEM_DEBOUNCE_MS) &&
        lastBinType >= 0 && lastBinType <= 3) {

        sendItemSorted(lastBinType);
        lastItemTime = now;
        lastBinType = -1;
    }
}

// ============================================================
// SEND ITEM_SORTED EVENT
// ============================================================

void sendItemSorted(int binIndex) {
    if (!isConnected) return;

    StaticJsonDocument<128> doc;
    JsonArray arr = doc.to<JsonArray>();
    arr.add("item_sorted");
    JsonObject data = arr.createNestedObject();
```

```
    data["type"] = binIndex;

    String message;
    serializeJson(doc, message);
    message = "42" + message;
    webSocket.sendTXT(message);

    const char* bins[] = {"Recyclable", "Wet", "Hazardous", "Dry"};
    Serial.printf("Sorted: %s\n", bins[binIndex]);
}

// ============================================================
// MOTOR CONTROL
// ============================================================

void setMotorSpeed(int pwmValue) {
    analogWrite(MOTOR_PIN, pwmValue);
    Serial.printf("Motor PWM: %d\n", pwmValue);
}
```

How to Use

1. Install required libraries in Arduino IDE
2. Change `WIFI_SSID`, `WIFI_PASSWORD`, and `SERVER_IP`
3. Upload to ESP32
4. Monitor via Serial (115200 baud)